

6. Metodologías ágiles

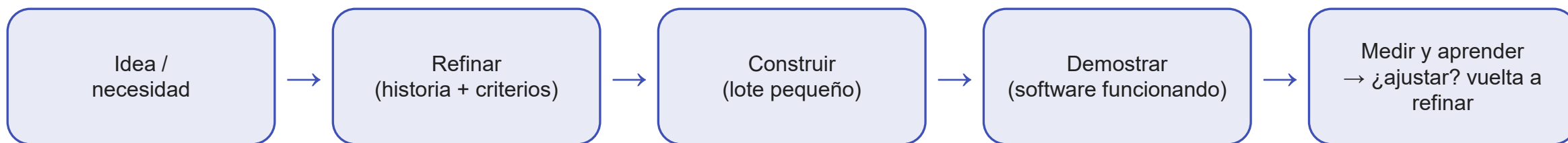
Manifiesto Ágil · Scrum · Kanban · XP · Lean · Tableros digitales

¿Qué son y por qué surgieron?

Las metodologías ágiles son una forma moderna de organizar el trabajo en equipo. Objetivo: entregar valor al usuario rápidamente, adaptarse al cambio y mejorar continuamente mediante ciclos cortos y retroalimentación constante. No son reglas rígidas: son una forma de pensar — priorizas lo que aporta valor, lo entregas pronto, ves qué funciona y ajustas el rumbo.

¿Por qué surgieron? Antes de los 2000 dominaba la cascada: análisis completo → diseño completo → implementación → pruebas. El cliente no veía nada hasta el final, los requisitos cambiaban pero el plan no, y los proyectos entregaban tarde, con más coste y sin resolver lo que se necesitaba. En 2001 un grupo de desarrolladores redactó el MANIFIESTO ÁGIL como alternativa.

Ciclo ágil: de idea a entrega



Cada vuelta comprueba si vamos bien antes de avanzar demasiado: descubrir un problema en la segunda iteración hace poco daño; descubrirlo en el mes diez es enorme.

6.1 Manifiesto Ágil — los cuatro valores

Cada valor compara dos cosas que importan, pero indica cuál pesa más cuando hay que elegir:

Se valora más...	...que	En la práctica
Personas e interacciones	Procesos y herramientas	Reuniones diarias breves, trabajo en parejas, decisiones tomadas por quien está más cerca del problema
Software funcionando	Documentación exhaustiva	Demos frecuentes al cliente, entregas pequeñas pero reales, medir el progreso por lo que funciona
Colaboración con el cliente	Negociación de contratos	El cliente participa, ayuda a definir criterios de aceptación y ve el producto en cada iteración
Respuesta al cambio	Seguir un plan	Re-priorizar según lo aprendido en cada entrega: no es un fallo, es el sistema funcionando bien

Ejemplo del ciclo: un equipo desarrolla una app de recetas. Primera iteración (2 semanas): pantalla principal y buscador. Los usuarios prefieren filtrar por ingredientes que ya tienen → la siguiente iteración prioriza ese filtro. Con seis meses de desarrollo a ciegas, ese descubrimiento habría llegado tardísimo.

6.1 Los 12 principios del Manifiesto

Los doce principios desarrollan los cuatro valores. Se agrupan en tres ideas:

Grupo	Principios	Contenido
Entrega temprana y frecuente	1, 3, 7	Satisfacer al cliente con entregas tempranas · entregar software funcionando a menudo (semanas, no meses) · medir el progreso por software funcionando
Colaboración y aprendizaje	2, 4, 6, 11, 12	Aceptar cambios incluso tarde · negocio y desarrollo trabajan juntos a diario · la conversación cara a cara es lo más eficiente · los mejores equipos son autoorganizados · el equipo reflexiona periódicamente y ajusta su comportamiento
Calidad y ritmo sostenible	5, 8, 9, 10	Personas motivadas y confianza en ellas · ritmo sostenible indefinidamente · atención continua a la excelencia técnica y al buen diseño · simplicidad: maximizar el trabajo NO hecho es esencial

6.2 Scrum — roles y artefactos

Scrum es el marco ágil más utilizado. Organiza el trabajo en SPRINTS (iteraciones de longitud fija, normalmente 2 semanas) con un objetivo claro. Al final de cada sprint hay algo entregable: no un diseño ni un documento, sino SOFTWARE QUE FUNCIONA.

Tres roles — añadir más jerarquía rompe la autoorganización

Rol	Responsabilidad principal
Product Owner	Decide QUÉ se construye y en QUÉ ORDEN según el valor que aporta al usuario
Scrum Master	Facilita el proceso, elimina obstáculos y ayuda al equipo a mejorar
Equipo de desarrollo	Decide CÓMO se construye; multidisciplinar y autoorganizado

Tres artefactos — las "listas" que mantiene el equipo

Artefacto	Qué es
Product Backlog	Todo lo que se quiere construir, ordenado por valor. Nunca está terminado: se ajusta según lo aprendido
Sprint Backlog	Lo que el equipo se compromete a hacer este sprint, más el objetivo del sprint
Incremento	El resultado del sprint: algo que funciona y que el cliente podría usar si se publicara

6.2 Scrum — eventos y Definition of Done

Evento	Duración típica	Para qué sirve
Sprint	2 semanas	Contenedor de todo lo demás; tiene un objetivo concreto
Sprint Planning	≈ 4 horas	El equipo elige qué hará este sprint y cómo lo abordará
Daily Scrum	15 minutos	Sincronización diaria: qué hice ayer, qué haré hoy, qué me bloquea
Sprint Review	≈ 2 horas	Demo al cliente: mostrar lo construido y recoger feedback
Retrospectiva	≈ 1,5 horas	El equipo mejora CÓMO trabaja, no qué construye

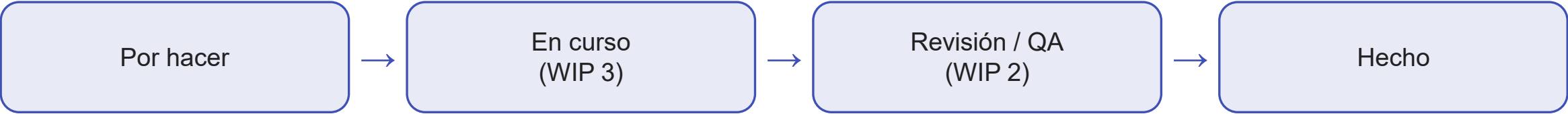
El Daily NO es un reporte al jefe: es una sincronización entre compañeros para coordinarse y detectar bloqueos. Si alguien lleva dos días bloqueado, el Daily es el momento de pedirlo.

Definition of Done (DoD) — qué significa "terminado de verdad"

El acuerdo del equipo que evita el "yo lo he hecho pero no está integrado/probado/documentado". DoD típico de clase: código integrado en la rama principal · pruebas automáticas en verde · revisión de otro compañero · documentación mínima actualizada. Si no cumple el DoD, no cuenta como terminado en la Review.

6.3 Kanban

Kanban no usa iteraciones fijas: gestiona un FLUJO CONTINUO — cuando terminas una tarea, empiezas la siguiente. Útil cuando el trabajo llega de forma irregular o hay que cambiar prioridades sin esperar al siguiente sprint. El nombre viene del japonés "tarjeta visual": Toyota usaba tarjetas físicas para controlar el trabajo en cada fase de la cadena de montaje.



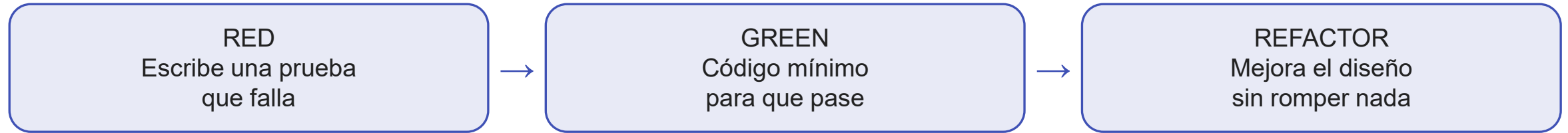
Regla	Qué significa
1. Visualizar el flujo	Todo el equipo ve el proceso completo: pendiente, en marcha, bloqueado. Si "Revisión" está llena, el equipo pausa tareas nuevas y ayuda a revisar
2. Políticas claras por columna	Reglas explícitas para avanzar: "pasa a Revisión solo con pruebas unitarias completas". Sin reglas, "En curso" significa cosas distintas para cada persona
Métrica	Qué mide
Lead Time	Tiempo total desde que se pide algo hasta que está Hecho
Cycle Time	Tiempo desde que se EMPIEZA una tarea hasta que se termina
Throughput	Número de tareas completadas por semana

6.4 XP (Extreme Programming)

XP se centra en la EXCELENCIA TÉCNICA: mientras Scrum organiza el equipo y el trabajo, XP se ocupa de cómo escribir el código. Si las prácticas técnicas son sólidas, el equipo puede cambiar el código con confianza y a cualquier ritmo.

Práctica	En qué consiste
TDD	Primero la prueba que falla, luego el código mínimo para que pase, luego mejorar el diseño sin romper nada
Pair programming	Dos personas, un teclado: driver (teclea) + navigator (piensa el diseño y detecta errores). Roles alternados cada 15-20 min. Errores detectados al momento; el conocimiento se distribuye
Refactorización continua	Mejorar el código sin cambiar su comportamiento (renombrar, eliminar duplicados, simplificar), en pasos pequeños tras pasar las pruebas. Las pruebas son la red de seguridad
Integración continua	Commits pequeños y frecuentes; en cada integración se compila, prueba y analiza. Las ramas longevas son el enemigo: cuanto más vive una rama, más difícil es integrarla
Propiedad colectiva	El código es del equipo, no de quien lo escribió: cualquiera puede mejorar cualquier parte. Requiere estándares compartidos; el formateador y el linter son los árbitros del estilo

6.4 XP — el ciclo TDD con ejemplo



```
// RED: el test falla porque la clase no existe aún
@Test
void totalConIVA() {
    assertEquals(121.0, Precio.conIVA(100.0, 0.21));
}

// GREEN: implementación mínima para que pase
class Precio {
    static double conIVA(double base, double iva) {
        return base * (1 + iva);
    }
}

// REFACTOR: añadimos redondeo sin romper el test
static double conIVA(double base, double iva) {
    return Math.round(base * (1 + iva) * 100.0) / 100.0;
}
```

6.5 Lean Software Development

Lean adapta al software los principios de fabricación de Toyota: identificar qué aporta valor al cliente y eliminar lo demás. En software, "desperdicio" es tiempo y esfuerzo que no llegan al usuario: cada vez que algo tarda, se espera, se repite o se construye sin que nadie lo pida.

Principio	En la práctica
Eliminar desperdicio	Quitar burocracia, aprobaciones innecesarias, funciones que nadie usa
Calidad desde el origen	Detectar y corregir cuanto antes; pruebas automatizadas desde el primer día
Aprender rápido	Entregas pequeñas + retroalimentación real → ajustar antes de invertir demasiado
Decidir tarde, entregar pronto	Sin decisiones irreversibles antes de tener información; entregar algo pequeño que dé datos reales
Respetar a las personas	Equipos con autonomía y confianza encuentran mejores soluciones que los microdirigidos
Optimizar el sistema completo	No optimices solo "tu parte": importa el tiempo total desde que se pide hasta que llega al usuario

Desperdicio	Cómo se manifiesta	Cómo reducirlo
Trabajo a medias	Ramas abiertas semanas, funciones a medias	Commits frecuentes, entregas pequeñas
Funcionalidad extra	Construir "por si acaso" sin que nadie lo pida	Solo lo que está en el backlog priorizado
Esperas	Esperar aprobación, otro módulo, el entorno	Automatizar, políticas claras, entornos self-service
Defectos y retrabajo	Bugs que vuelven, código que se reescribe	TDD, revisiones de código, pruebas automáticas
Cambios de tarea	Seis tareas "en curso" a la vez por persona	Limitar el WIP; terminar antes de empezar

6.6 Herramientas: tableros digitales

Un tablero digitaliza el flujo de Kanban o Scrum y lo hace accesible a todo el equipo, en remoto y desde cualquier dispositivo:

Herramienta	Descripción	Cuándo usarla	Curva de aprendizaje
Jira	El estándar profesional: flujos a medida, permisos por rol, tareas conectadas con versiones del software	Equipos medianos/grandes con procesos formales o trazabilidad tarea-versión	Alta: potente pero compleja de configurar
Trello	Tablero muy sencillo de listas y tarjetas, con checklists, fechas límite y adjuntos. Sin configuración inicial	Proyectos pequeños, equipos que empiezan con Kanban, trabajos de clase	Mínima: en diez minutos funciona
GitHub Projects	Integrado en GitHub: cada tarjeta se vincula a un issue (tarea o bug del repositorio) o a una pull request	Cuando ya usas GitHub para el código: misma plataforma para código y tareas	Baja si ya conoces GitHub

Políticas de columna (sin reglas escritas, cada persona interpreta el tablero a su manera)	
Columna	Condición para entrar
En curso	Hay capacidad libre y la tarea está bien descrita (se sabe qué hacer y cuándo estará terminada)
Revisar / QA	El código está hecho, las pruebas pasan y hay una Pull Request abierta

Revisión semanal del tablero (15 min): ¿tarjetas atascadas? ¿columnas desbordadas? Una acción de mejora por semana basta. WIP habitual: una tarea activa por persona.